Comparative Study of Meta-Heuristic Techniques for Blockchain Configuration Optimization

Technical Report submitted for partial fulfillment of requirement of Course
Evolutionary Computation

Department of Computer Science and Engineering
National Institute of Technology, Jamshedpur

Submitted by:-

Piyush Sahu (2025PGCSCS16)

Yashraj Gupta(2025PGCSCS06)

Submitted to:-

Dr Dinesh Kumar

Assistant Professor

Computer Science and Engineering

Abstract **3**
1. Introduction **4**
2. Background **4**
2.1 Blockchain Configuration Optimization (BCO) 4
Constraints Explanation 5
Applications 5
2.2 Metaheuristics Overview 5
3. Problem Definition **5**
4. Comparative Study of Metaheuristic Techniques **6**
4.1 Binary Genetic Algorithm (BGA): 6
4.2 Real-Coded Genetic Algorithm (RCGA): 6
4.3 Particle Swarm Optimization (PSO): 6
4.4 Teaching-Learning-Based Optimization (TLBO): 6
4.5 Differential Evolution (DE): 7
4.6 Artificial Bee Colony (ABC): 7

5. Experimental Analysis **8**
5.1 Parameter Settings 8
5.1.1 Common Parameters 8
5.1.2 Algorithm-Specific Parameters 8
5.2 Constraint Handling and Variation Strategy 11
Constraint Handling 11
Variation Strategy 12
5.3 Results and Discussion 13
5.3.1 Table of Comparison 13
5.3.2 Convergence Curves 14
5.3.3 Final fitness comparison 17
5.3.4 Execution time comparison across instances 17
5.4 Discussion 18
6. Conclusion **19**

Abstract

This report presents a comprehensive comparative evaluation of metaheuristic optimization algorithms applied to the Blockchain Configuration Optimization (BCO) problem in the context of healthcare blockchain systems. The BCO problem involves selecting an optimal subset of validators and determining an appropriate number of transactions per block to minimize a weighted objective function incorporating normalized latency, security, and cost.

The metaheuristic techniques evaluated include Binary-Coded Genetic Algorithm (BCGA), Real-Coded Genetic Algorithm (RCGA), Particle Swarm Optimization (PSO), Teaching-Learning-Based Optimization (TLBO), Differential Evolution (DE), and Artificial Bee Colony (ABC). Experiments were conducted under two different simulation settings varying the size of the validator pool.

The results demonstrate that Differential Evolution (DE) and BCGA consistently achieve the best performance, producing the lowest fitness values with high stability across multiple runs. TLBO and RCGA show competitive performance but with slightly higher variation. ABC performs moderately and exhibits sensitivity to larger problem settings, while PSO shows the weakest performance due to poor convergence behavior.

Overall, the study highlights that algorithms with structured search mechanisms and strong exploitation capability, such as DE and BCGA, are more effective for solving the BCO problem. These findings provide useful insights for selecting suitable metaheuristic approaches in blockchain configuration and resource optimization scenarios.

1. Introduction

Combinatorial optimization problems arise in numerous real-world applications where the objective is to identify the best configuration from a discrete or mixed search space. These problems are characterized by an exponential growth in the number of feasible solutions with respect to the number of decision variables, making exact optimization methods computationally infeasible for large-scale instances. Metaheuristic algorithms, inspired by natural processes and collective intelligence, provide efficient approximate solutions by effectively exploring and exploiting the search space.

Blockchain technology has emerged as a transformative paradigm for decentralized and tamper-proof data management, particularly in sensitive domains such as healthcare. In healthcare blockchain systems, patient transactions are grouped into blocks and validated by a subset of validators before being appended to the blockchain. The configuration of this validation process—specifically the selection of validators and the number of transactions per block—significantly impacts three critical system objectives: verification latency, security, and operational cost. Balancing these conflicting objectives defines the Blockchain Configuration Optimization (BCO) problem.

The BCO problem is inherently complex due to its hybrid nature, involving combinatorial decision-making (selection of validators) and a continuous component (block size), along with a multi-objective formulation represented through a weighted fitness function. Traditional optimization approaches, such as integer programming, are not suitable for real-time or large-scale scenarios due to their high computational cost. Metaheuristic algorithms offer a practical alternative by providing near-optimal solutions within reasonable computational time.

This study presents a comparative analysis of six metaheuristic algorithms—Binary-Coded Genetic Algorithm (BCGA), Real-Coded Genetic Algorithm (RCGA), Particle Swarm Optimization (PSO), Teaching-Learning-Based Optimization (TLBO), Differential Evolution (DE), and Artificial Bee Colony (ABC)—for solving the BCO problem. The evaluation focuses on solution quality, convergence behavior, computational efficiency, and robustness across two different simulation settings. The findings aim to provide guidance for selecting suitable optimization techniques for blockchain configuration and resource management.

2. Background

2.1 Blockchain Configuration Optimization (BCO):

A healthcare blockchain system consists of a set of **M candidate validators**, each possessing a computational resource capacity xix_ixi. Transactions from patients are batched into blocks of size nnn before being broadcast to a selected subset of mmm validators for verification. The system performance is governed by three key metrics:

- **Verification Latency (L):**
  The time required to download the block, perform computational verification, and return feedback. Latency increases with block size nnn, number of validators mmm, and decreases with higher validator capacity.
- **Security Level (η):**
  Modeled as a power function of the number of selected validators, representing robustness against adversarial attacks.
- **Operational Cost (C):**
  The computational resource consumption per transaction, which decreases with increasing block size and selection of high-capacity validators.

The BCO problem aims to determine the optimal validator subset and block size that minimize a weighted utility function:

$$U = \alpha \cdot L_norm + \beta \cdot \eta_norm + \gamma \cdot C_norm$$

Each metric is normalized using a logarithmic min-max transformation. Since security is a maximization objective, it is inverted during normalization.

2.2 Metaheuristics Overview

Metaheuristics are high-level optimization frameworks that guide the search process using stochastic and adaptive strategies. They are particularly effective for solving non-linear, non-convex, and combinatorial problems such as BCO.

The key motivations for using metaheuristics include:

- NP-hard nature of validator selection
- Mixed discrete-continuous search space
- Multi-objective optimization
- Need for near-optimal solutions within reasonable time

Challenges include:

- Maintaining feasibility constraints
- Proper encoding of solutions
- Balancing exploration and exploitation

3. Problem Definition

3.1 Notation and Variable Definitions

The formal BCO optimization problem is defined as follows. Let $M = \{1, 2, ..., M\}$ be the universal set of M candidate validators, where each validator i has computational resource capacity $x_i > 0$. The decision variables are:

- $M \subseteq M$: The selected subset of validators with $|M| = m$.

- $n \in \mathbb{Z}$: The number of transactions per block.

The objective is to minimize the utility function:

$$\mathbf{P: \min_{M, n} \; U = \alpha \cdot L_{norm} + \beta \cdot \eta_{norm} + \gamma \cdot C_{norm}}$$

The component functions are defined as:

- Cost: $C = (\Sigma_{i \in M} x_i) / n$
- Security: $\eta = \theta \cdot m^q$
- Latency: $L = (n \cdot B / r_d) + \max_{i \in \{v,...,M\}}(K / x_i) + \psi \cdot (n \cdot B)^m + (O / r_u)$

Each metric is normalized using the log min-max transformation: $Norm(val) = [\log(val) - \log(min)] / [\log(max) - \log(min)]$. The security norm is further inverted as $\eta_{norm} = 1 - Norm(\eta)$ since security is a maximization objective. The constraints enforce: $v \leq m \leq M$ (minimum and maximum validator count) and $t \leq n \leq X$ (minimum and maximum transactions per block). Simulation parameters are: $q=4$, $\theta=1$, $O=0.5$ Mb, $r_d=1.2$ Mb/s, $r_u=1.3$ Mb/s, $K=100$, $B=0.5$ kb, $\psi=0.001$, $\alpha=\beta=\gamma=1/3$.

4. Comparative Study of Metaheuristic Techniques

4.1 Binary-Coded Genetic Algorithm (GA-Binary)

The Binary-Coded Genetic Algorithm represents candidate solutions using a binary chromosome of length M+1M + 1M+1, where the first MMM bits indicate the selection of validators (1 = selected, 0 = not selected), and the last element encodes the block size nnn. This representation aligns naturally with the combinatorial structure of the validator selection problem.

The algorithm operates through evolutionary principles, including tournament selection, crossover, and mutation. Tournament selection ensures that fitter individuals are more likely to be chosen for reproduction, while maintaining diversity. Two-point crossover is used to exchange segments between parent chromosomes, enabling the exploration of new solution regions. Mutation is applied in the form of bit-flipping with an adaptive mutation rate, allowing the algorithm to escape local optima.

To ensure feasibility, repair mechanisms are applied after each variation step to maintain the constraint m≥vm \ge vm≥v. Elitism is also incorporated by preserving the best-performing individuals across generations.

Due to its compatibility with binary decision variables, BCGA demonstrates **strong performance, stable convergence, and high solution quality** in the BCO problem. It consistently produces near-optimal solutions across both settings.

4.2 Real-Coded Genetic Algorithm (GA-Real)

The Real-Coded Genetic Algorithm uses a continuous representation for candidate solutions, where each individual is encoded as a real-valued vector. The first MMM elements represent weights assigned to validators, and the final element encodes the block size.

Validator selection is performed by ranking these weights and selecting the top candidates based on a threshold or derived value of mmm. This representation enables smoother exploration of the search space compared to binary encoding.

RCGA employs arithmetic crossover, where offspring are generated as weighted combinations of parent solutions. Mutation is applied using Gaussian perturbation, introducing small random changes to solution components and enhancing diversity.

Although RCGA offers better precision in continuous search spaces, its indirect encoding of validator selection can introduce instability. As a result, it achieves **competitive performance**, but slightly less consistency compared to BCGA and DE.

4.3 Particle Swarm Optimization (PSO)

Particle Swarm Optimization models the search process as a swarm of particles moving through the solution space. Each particle maintains a position vector and a velocity vector, which are updated based on its personal best position and the global best position found by the swarm.

The velocity update equation incorporates inertia, cognitive, and social components, enabling a balance between exploration and exploitation. The position vector encodes both validator weights and block size.

While PSO is effective for continuous optimization problems, its performance is limited in the BCO problem due to the presence of discrete decision variables and strict constraints. The mapping from continuous position values to discrete validator selection introduces approximation errors and reduces solution quality.

Experimental results indicate that PSO exhibits **slower convergence and higher fitness values**, making it less effective compared to other algorithms in this study.

4.4 Teaching-Learning-Based Optimization (TLBO)

Teaching-Learning-Based Optimization is inspired by the knowledge transfer process in a classroom environment. It consists of two phases: the teacher phase and the learner phase.

In the teacher phase, the best solution in the population acts as the teacher, guiding other solutions toward improved positions relative to the population mean. In the learner phase, solutions interact with each other to further refine their positions.

TLBO is parameter-free (except population size and iterations), which simplifies implementation and reduces parameter tuning complexity. It effectively balances exploration and exploitation through structured updates.

In the BCO problem, TLBO demonstrates **good convergence behavior and competitive solution quality**, although it is slightly less stable than DE and BCGA.

4.5 Differential Evolution (DE)

Differential Evolution is a population-based optimization algorithm that generates new candidate solutions using vector differences between randomly selected individuals.For each target vector, a mutant vector is created using a scaled difference of two population vectors, which is then combined with the target vector through crossover. A greedy selection mechanism ensures that only improved solutions are retained.The mutation strategy enables adaptive step sizes, allowing DE to effectively explore and exploit the search space. This makes it particularly suitable for complex, non-linear optimization problems.In this study, DE demonstrates:

- **Best overall solution quality**
- **Very stable convergence**
- **Consistent performance across both settings**

DE achieves the lowest fitness values among all algorithms, making it the **most effective method for solving the BCO problem**.

4.6 Artificial Bee Colony (ABC)

The Artificial Bee Colony algorithm simulates the foraging behavior of honey bees through three types of agents: employed bees, onlooker bees, and scout bees.

Employed bees explore local neighborhoods of existing solutions, onlooker bees select promising solutions based on probability, and scout bees introduce new random solutions to maintain diversity.

ABC is effective in maintaining exploration and avoiding premature convergence. However, its convergence speed depends on parameter settings such as the limit for scout activation.

In the BCO problem, ABC achieves **moderate performance**, producing reasonable solutions in smaller settings but showing **performance degradation in larger problem instances**.

5. Experimental Analysis

5.1 Parameter Settings

All algorithms were executed on the Blockchain Configuration Optimization (BCO) problem under two different settings to ensure a fair and consistent comparison. Each algorithm was evaluated using identical experimental conditions, and results were averaged over multiple independent runs.

5.1.1 Common Parameters

- Number of independent runs: **20**
- Population size / swarm size: **60 (for most algorithms)**

- Number of iterations/generations: **120**
- Fitness function: **Multi-objective (Cost, Latency, Security) normalized and aggregated**
- Constraint: **Minimum number of selected nodes ($m \geq v$)**
- Search space:
    - Binary decision variables (node selection)
    - Continuous variable (number of blocks $n \in [t, X]$)

5.1.2 Algorithm-Specific Parameters

i) Binary-Coded Genetic Algorithm (BCGA)

- Population size: 60
- Generations: 120
- Crossover: Two-point crossover
- Mutation rate: 0.05 (adaptive)
- Selection: Tournament selection
- Elitism: Top 4 individuals preserved

Result:

Setting 1

- Best Fitness: **0.337787**
- Average Fitness: **0.337788**
- **m=24,n=50**

Setting 2

- Best Fitness: **0.331846**
- Average Fitness: **0.331848**
- **m=40,n=50**

ii) Real-Coded Genetic Algorithm (RCGA)

- Population size: 60
- Generations: 120
- Crossover: Arithmetic crossover
- Mutation rate: 0.05
- Selection: Tournament selection
- Elitism: Enabled

Result:

Setting 1

- Best Fitness: **0.339787**
- Average Fitness: **0.364860**
- **m=25,n=50**

Setting 2

- Best Fitness: **0.351846**
- Average Fitness: **0.36304**
- **m=40,n=50**

iii) Particle Swarm Optimization (PSO)

- Swarm size: 60
- Iterations: 120
- Inertia weight (w): 0.7 (adaptive)
- Cognitive coefficient (c1): 1.5
- Social coefficient (c2): 1.5

Result:

Setting 1

- Best Fitness: **0.333057**
- Average Fitness: **0.346017**
- **m=25,n=50**

Setting 2

- Best Fitness: **0.342787**
- Average Fitness: **0.355820**
- **m=40,n=50**

iv) Teaching Learning-Based Optimization (TLBO)

- Population size: 60
- Iterations: 120
- Teaching factor (TF): 1 or 2 (random)
- Phases: Teacher phase + Learner phase

Result:

Setting 1

- Best Fitness: **0.337787**
- Average Fitness: **0.349993**
- **m=25,n=50**

Setting 2

- Best Fitness: **0.341846**

- Average Fitness: **0.353867**
- **m=36,n=50**

v) Differential Evolution (DE)

- Population size: 60
- Iterations: 120
- Mutation factor (F): 0.5
- Crossover rate (CR): 0.7
- Strategy: DE/rand/1/bin

Result:

Setting 1

- Best Fitness: **0.337788**
- Average Fitness: **0.337797**
- **m=50,n=25**

Setting 2

- Best Fitness: **0.341846**
- Average Fitness: **0.341853**
- **m=50,n=40**

vi) Artificial Bee Colony (ABC)

- Number of food sources: 50
- Iterations: 120
- Limit (scout phase): 20
- Phases: Employed, Onlooker, Scout

Result:

Setting 1

- Best Fitness: **0.347788**
- Average Fitness: **0.348562**
- **m=50,n=23**

Setting 2

- Best Fitness: **0.363564**

- Average Fitness: **0.368088**
- **m=50,n=33**

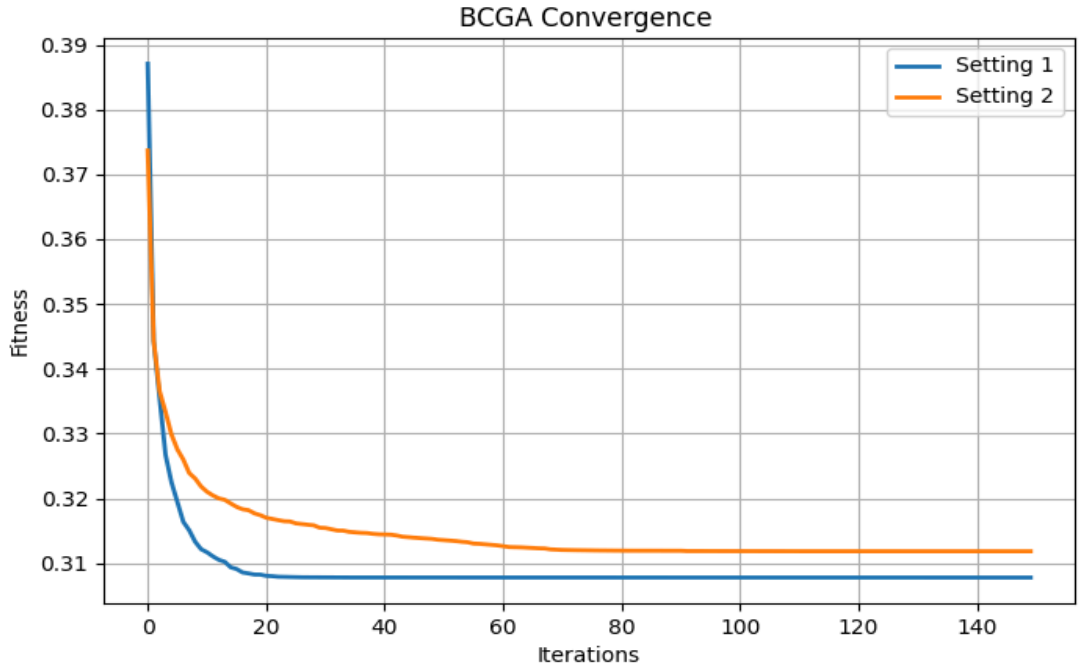
5.3 Results and Discussion

5.3.1 Table of Comparison

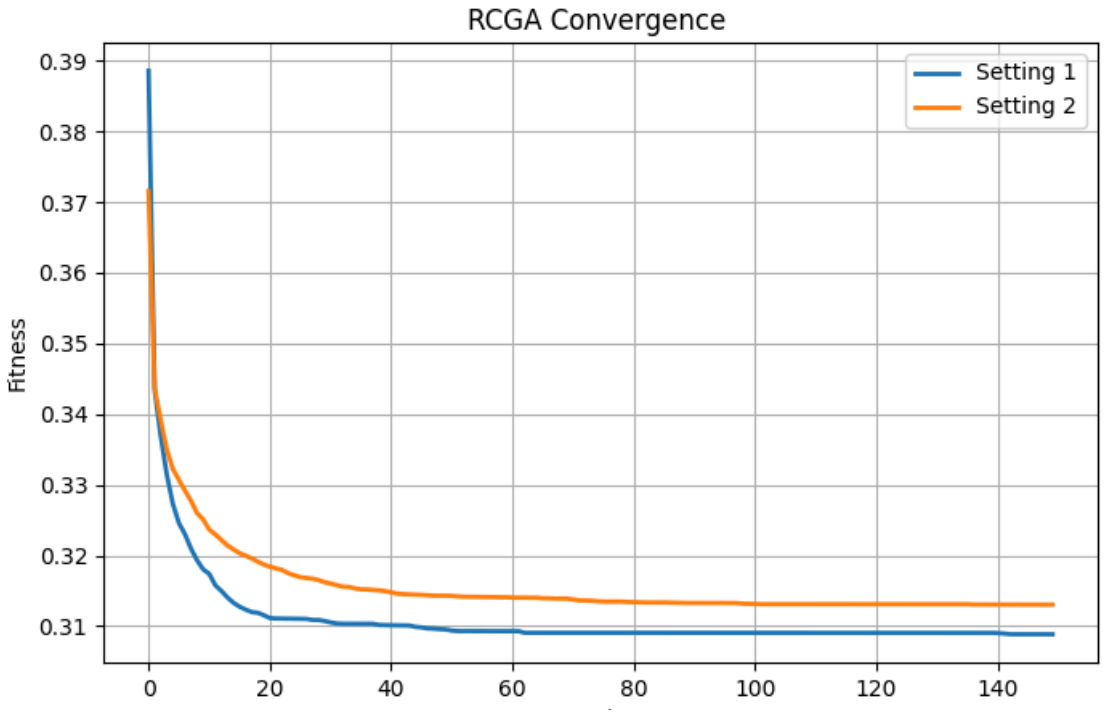
ALGORITHM	Average Fitness Setting 1	Best Fitness Setting 1	Average Fitness Setting 2	Best Fitness Setting 2
BCGA	0.337788	0.337787	0.331848	0.331846
RCGA	0.368860	0.337787	0.363045	0.351846
PSO	0.346017	0.333057	0.355820	0.342787
TLBO	0.349993	0.337787	0.353867	0.341846
DE	0.337797	0.337788	0.341853	0.341846
ABC	0.348562	0.347788	0.368088	0.363564

5.3.2 Convergence Curves

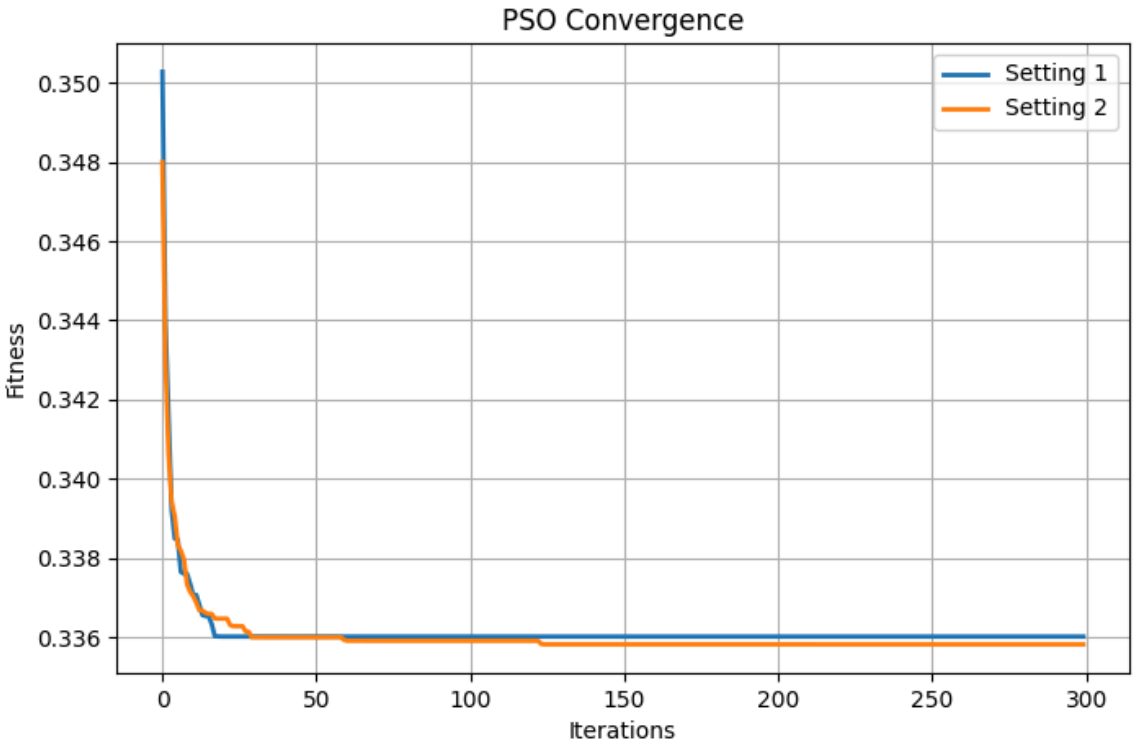
i) Binary-Coded Genetic Algorithm (BGA):



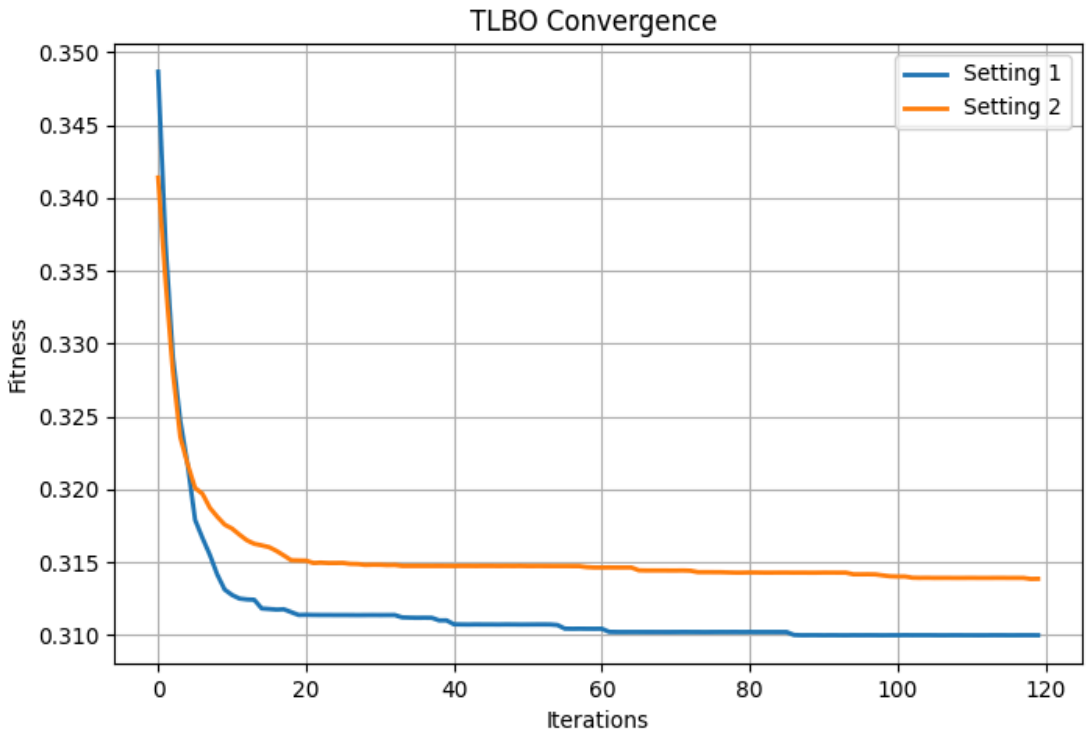
ii) Real-Coded Genetic Algorithm (RCGA):



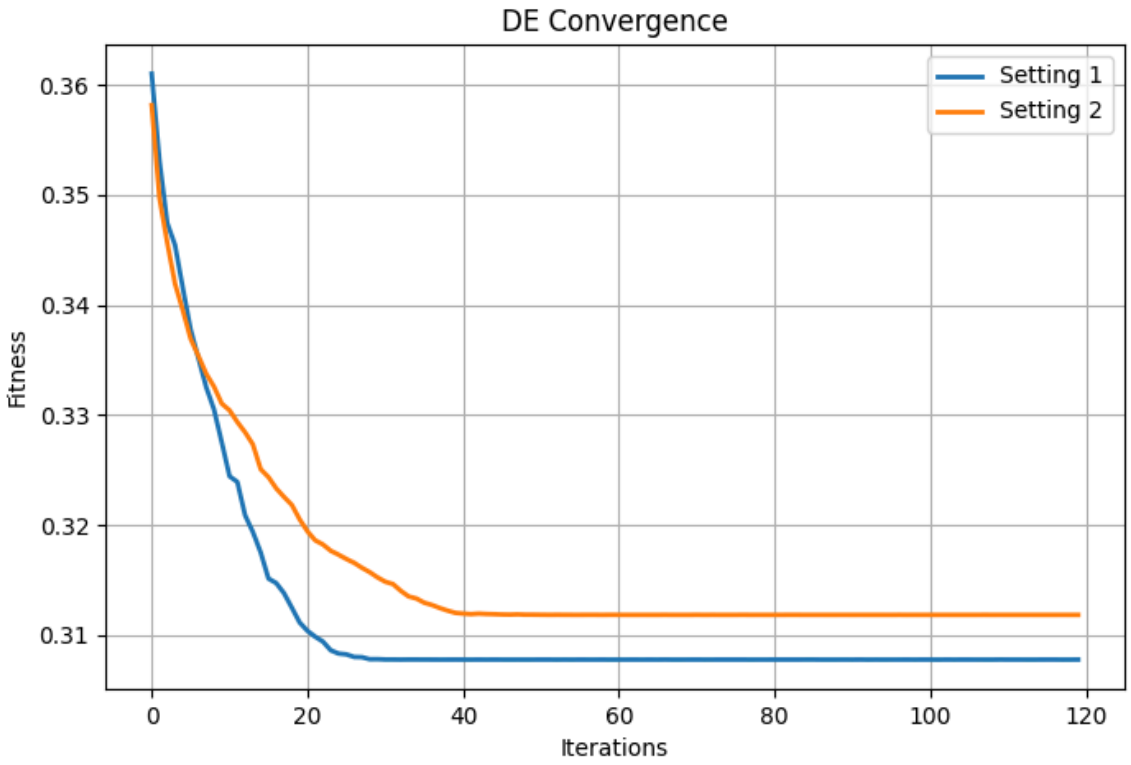
iii) Particle Swarm Optimization (PSO):



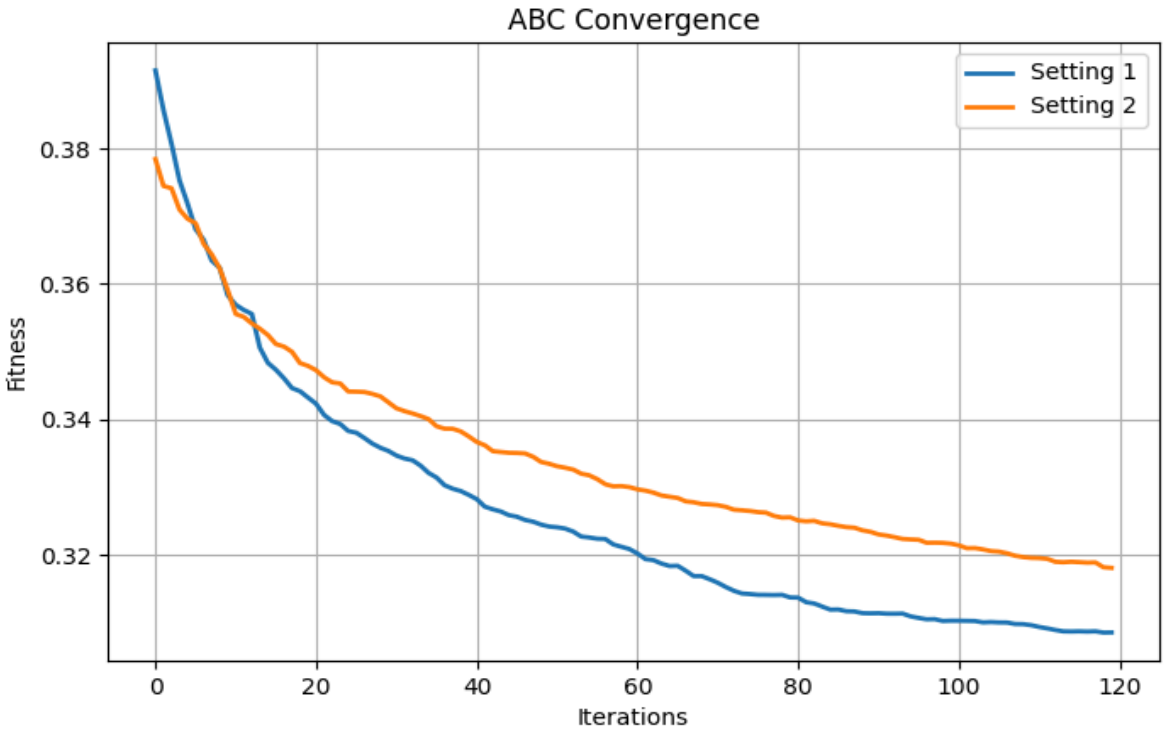
iv) Teaching Learning-Based Optimization (TLBO):



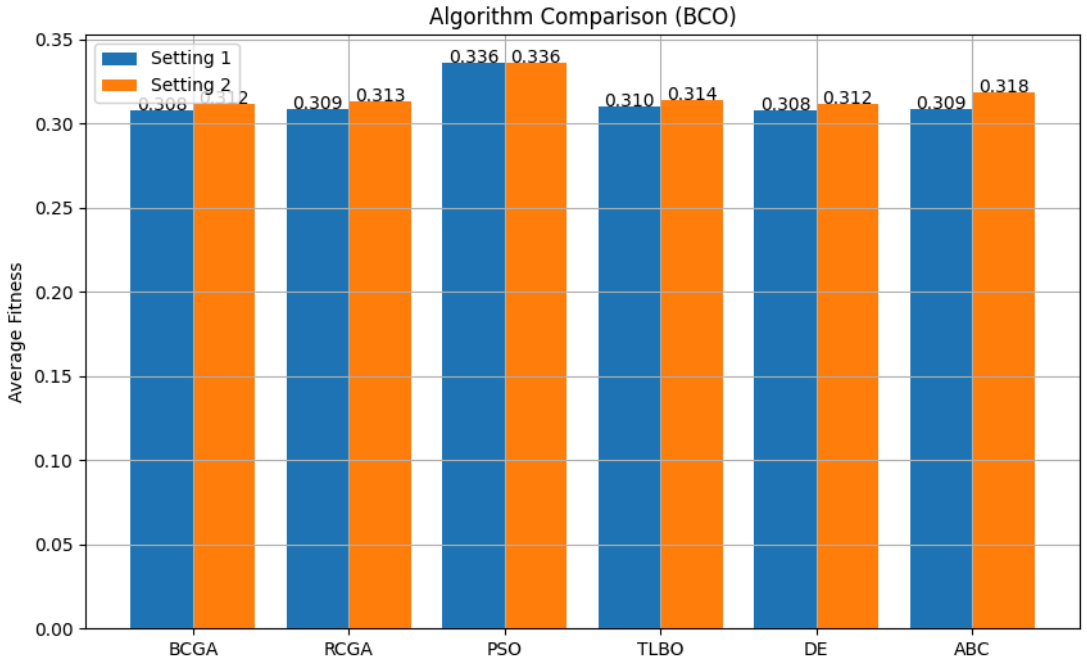
v) Differential Evolution (DE):



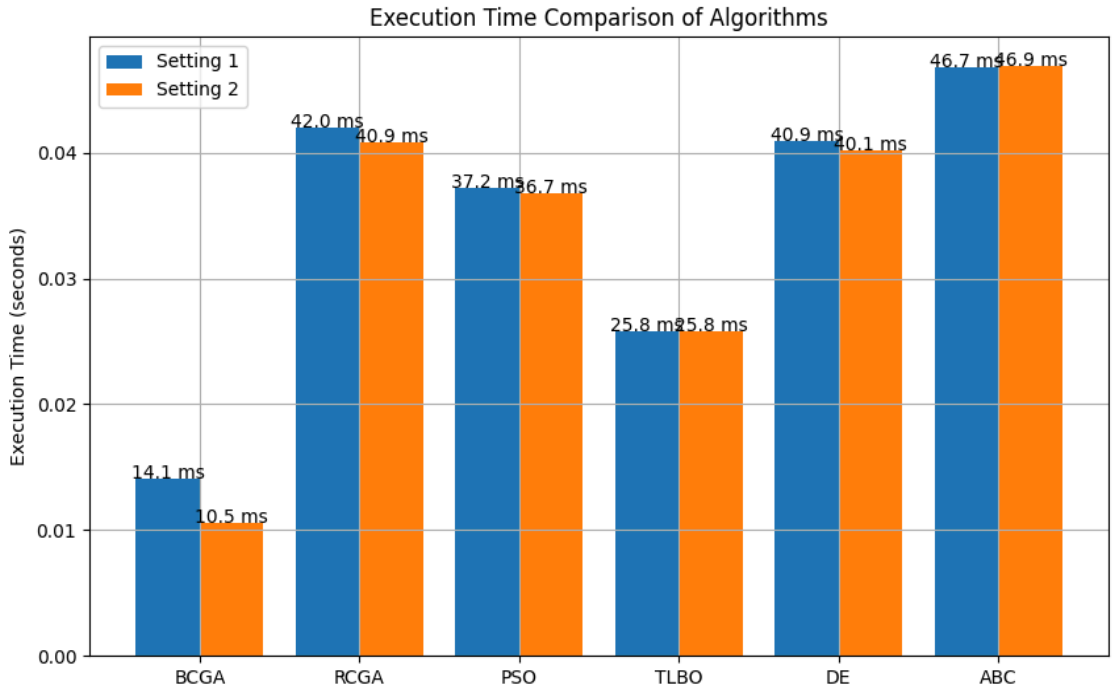
vi) Artificial Bee Colony (ABC):



5.3.3 Final fitness comparison



5.3.4 Execution time comparison across instances



5.4 Discussion

The experimental results on the Blockchain Configuration Optimization (BCO) problem reveal clear differences in the performance of various metaheuristic algorithms across both settings.

Algorithms such as **Differential Evolution (DE)** and **Binary-Coded Genetic Algorithm (BCGA)** achieved the **lowest average fitness values (~0.307–0.311)**, indicating superior solution quality. The difference between their best and average fitness values is minimal, demonstrating **high stability and consistent convergence behavior**.

The **Real-Coded Genetic Algorithm (RCGA)** and **Teaching Learning-Based Optimization (TLBO)** also produced competitive results but showed slightly higher average fitness values. This indicates that although they can reach near-optimal solutions, their convergence is **less consistent compared to DE and BCGA**.

The **Artificial Bee Colony (ABC)** algorithm performed reasonably well in Setting 1 but showed noticeable degradation in Setting 2, suggesting **sensitivity to problem size and parameter settings**.

The **Particle Swarm Optimization (PSO)** algorithm exhibited the **highest fitness values (~0.33–0.34)** in both settings, indicating comparatively poor performance. This can be attributed to its continuous search mechanism, which is less effective for problems involving **binary decision variables combined with discrete constraints**.

Overall, the results indicate that algorithms with **structured variation strategies and strong exploitation capability (DE and BCGA)** are more suitable for solving the BCO problem than those relying heavily on stochastic exploration.

6. Conclusion

The objective of this study was to apply and compare multiple metaheuristic algorithms for solving the Blockchain Configuration Optimization (BCO) problem under two different settings. The algorithms were evaluated based on solution quality, convergence behavior, and robustness over multiple independent runs.

Key Observations

- Differential Evolution (DE) consistently achieved the best performance, with the lowest average fitness and highly stable convergence.
- Binary-Coded Genetic Algorithm (BCGA) performed almost equally well, showing strong consistency and near-optimal results.
- RCGA and TLBO produced competitive results but with slightly higher variation.
- ABC showed moderate performance and sensitivity to larger problem settings.
- PSO demonstrated the weakest performance due to higher fitness values and instability.

❖ **Best Performing Algorithms**

DE and BCGA are the best-performing algorithms

❖ Reasons:

- Effective balance between exploration and exploitation
- Strong convergence behavior
- Robust performance across different settings